

Quantum neural network-based inverse kinematics of a six-jointed industrial robotic arm

Mehdi Fazilat^{*}, Nadjat Zioui

Université du Québec à Trois-Rivières, 3351 Bd des Forges, Trois-Rivières QC G8Z 4M3, Canada

ARTICLE INFO

Keywords:

Robotic arms
Inverse kinematics
Machine learning
Artificial neural networks
Multi-layered perceptron
Activation function
Quantum computing
Quantum-inspired neural networks
Singularity avoidance

ABSTRACT

This research examines the potential of quantum-inspired neural networks (QNNs) for solving the inverse kinematics of robotic arms, focusing on the six-degree-of-freedom ABB IRB140 robot. Traditional inverse kinematics approaches face challenges such as non-unique solutions and computational complexity, especially with increasing degrees of freedom. While artificial neural networks (ANNs) have shown promise, they require further improvements, particularly in terms of quantum computing integration. This study introduces a quantum-inspired activation function to multi-layer perceptron neural networks. We compared ANNs and QNNs with and without singularity avoidance, finding that QNNs significantly outperformed ANNs in mean absolute error (MAE), achieving a 15.60 % lower MAE in singularity-free models and a 16.67 % lower MAE in singularity-avoidance models. The QNNs demonstrated superior precision, with a position error of 1.64 mm and an orientation error of 0.00179 radians when avoiding singularities. These results highlight the potential of QNNs to enhance the precision, efficiency, and performance of robotic arm manipulation. Quantum computing offers advantages including parallelism, quantum entanglement, and quantum annealing, which contribute to the QNNs' superior performance. Overall, this study represents a practical contribution to robotics and quantum computing, paving the way for future research into applying quantum principles to neural network models for robotics.

1. Introduction

The inverse kinematics of robotic arms are crucial for determining the precise positions and movements required to execute complex tasks. As the degrees of freedom in robotic arms increase, traditional methods for solving inverse kinematics typically encounter significant challenges, such as non-unique solutions and increasing computational complexity. These issues highlight the need for innovative approaches that can simplify and enhance the accuracy of inverse kinematics computations in robotics. Recent advancements in robotics have increasingly applied machine learning techniques, particularly artificial neural networks (ANNs), to address these challenges. ANNs have shown promise in solving inverse kinematics problems due to their ability to learn from data and generalise to new situations. However, further improvements are still required in classical ANNs in terms of computational efficiency and their ability to handle high-dimensional data, thus necessitating further model development and innovation.

Quantum-inspired neural networks (QNNs) offer a promising solution for addressing these challenges by integrating principles from

quantum computing. Quantum computing leverages phenomena such as parallelism, quantum entanglement, and quantum annealing, which can significantly improve the computational efficiency and accuracy of neural networks. By incorporating quantum principles into neural networks, QNNs can potentially outperform classical ANNs, particularly in tasks requiring high precision and efficiency. In this study, the performance of QNNs is evaluated in a practical application of solving the inverse kinematics of a six-degree-of-freedom articulated industrial robotic arm. By introducing a quantum-inspired activation function into multi-layered perceptron (MLP) neural networks, this study aims to improve the efficiency and performance of classical neural networks by integrating advancements from the emerging field of quantum computing, thus providing a more robust and efficient solution for robotic arm kinematics and examining the potential of applying QNNs to the field of robotics in terms of enhancing the precision and efficiency of robotic arm manipulations.

In recent years, there has been a growing trend in robotics of applying neural network methodologies to improve the precision and efficiency of solving kinematic equations and controlling manipulator

^{*} Corresponding author.

E-mail addresses: mehdi.fazilat@uqtr.ca (M. Fazilat), nadjat.zioui@uqtr.ca (N. Zioui).

<https://doi.org/10.1016/j.robot.2025.105123>

Received 27 March 2024; Received in revised form 5 April 2025; Accepted 5 July 2025

Available online 6 July 2025

0921-8890/© 2025 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

robots. For example, Oliveira et al. introduced the Pre-Multiplication method to simplify the inverse kinematics problem and enhance singularity robustness [1]. Theofanidis et al. showcased the application of multilayer feed-forward neural networks on the Sawyer Robotic Arm, demonstrating machine learning-based model-agnostic control schemes [2]. Additionally, concentric tube continuum robots have improved by using two artificial neural networks with ReLU activation functions, enhancing their approximation accuracy [3]. Integrating current robotic arm states into the network's input has also been confirmed as a beneficial approach for improving motion control accuracy and effectively handling redundant degrees of freedom [4]. Another study utilised Gaussian mixture models within a hierarchical neural network structure to improve the accuracy and flexibility of robotic arm motions [5]. Machine learning frameworks have also been used to optimise inverse kinematics solutions for redundant manipulators [6], while sim-to-real transfer learning aims to bridge simulation and real-world applications, especially for soft robots [7]. Adar's multi-layered feed-forward ANN model, incorporated with a proportional-integral control algorithm, demonstrated superior tracking ability and reduced control error for a robotic arm [8]. Hargis et al. achieved millimetre precision in tool space positioning and reduced computation time by training an ANN for a serial manipulator [9]. Additionally, Csiszar et al. showcased the flexibility and effectiveness of neural networks in robotic arm inverse kinematics and calibration using supervised learning [10]. Furthermore, Srisuk et al. developed custom neural networks using forward kinematic equations to simplify the control of robotic arm joint angles [11]. Aparanji et al. also introduced a multi-layered Auto Resonance Network architecture for path planning, avoiding non-linear inverse kinematic computations [12].

The combination of quantum computing with ANNs improves model performance by leveraging quantum mechanics properties such as superposition, entanglement, and quantum parallelism [13–15]. This integration leads to quicker training times, reduced computational complexity, and enhanced accuracy in ANN models [16,17]. QNNs, which are based on quantum computing principles, can be potentially used to enhance the control and kinematics of robots. Recent studies of QNNs have made various advances that are relevant to robotics; for example, Nguyen et al. found that QNNs are more efficient and provide significantly improved performance compared to classical neural networks [18]. Gupta et al. demonstrated the potential of quantum algorithms to optimise execution times and data storage, which can lead to advancements in robotics [19]. Zidan et al. introduced a quantum perceptron neural network, thereby enhancing computational efficiency for complex problems [20]. Kutvonen et al. explored the memory capacity of a quantum reservoir computer, which is also relevant for efficient QNN design in robotics [21]. In addition, Singh and Rajput highlighted the role of quantum entanglement in enhancing computational interconnectivity in robotic control systems [22].

Recent studies have applied quantum computing and quantum-inspired methods to enhance robotic arm kinematics, including a novel quantum model for forward kinematics leveraging quaternions and Pauli gates to improve positioning accuracy [23,24]. Further research has extended to representing quantum spins in various coordinate systems for rigid body orientation, thus showcasing the adaptability of quantum computing for robotics applications [25]. A QNN-based control design for a robotic manipulator was presented that integrates quantum computing with neural networks for enhanced precision and efficiency [26]. Learning inverse kinematics with a quaternion neural network exemplifies an innovative approach whereby quantum computing can be leveraged in robotics [27]. In addition, a proposed quantum-behaving particle swarm algorithm represents a cutting-edge solution for inverse kinematics challenges in complex robotic systems [28]. Furthermore, a proposed inverse solution algorithm for a robotic arm based on the MQACA-RBF network involved integrating quantum-inspired algorithms with neural networks to achieve high accuracy and efficiency [29].

A practical approach to developing QNNs involves integrating quantum computing with the activation function in ANNs to produce quantum mechanics-inspired activation functions. Parisi et al. introduced QReLU and m-QReLU, quantum activation functions that can improve convolutional neural networks (CNNs) in medical diagnostics by utilising entanglement and superposition, thereby solving the 'dying ReLU' problem and improving disease diagnosis accuracy [30]. Dong et al. presented a hybrid QNN with a quantum activation function, enhancing air quality prediction by integrating quantum techniques into classical CNNs with extended short-term memory networks, effectively capturing correlations while addressing gradient issues and the 'dying ReLU' problem [31]. Daskin proposed a QNN using a periodic activation function based on cosine values, which leverages the superposition principle to achieve exponential speed-up and high precision with fewer qubits and gates [32]. Li et al. developed a hybrid quantum-classical CNN with quantum-inspired activation functions based on cosine series, improving training convergence and attribute selection efficiency on the MNIST database [33]. She et al. introduced a novel ANN framework based on quantum interference principles – this framework incorporates a quantum activation function to enhance the network's universal approximation capabilities and effectively track high-frequency disturbances for space manipulators [34]. Additionally, Gandhi and McGinnity proposed a new neural network approach based on quantum interference principles to enhance the performance of adaptive controllers for robotic hands. Their proposed network utilises a quantum activation function, thus providing superior tracking performance and robustness [35].

Overall, quantum activation functions have been investigated across different disciplines, with the outcomes of various studies indicating meaningful benefits to using these approaches, particularly in robotic applications. However, a considerable gap remains in terms of using quantum computing to develop quantum activation functions to address critical problems such as inverse kinematics. Given the significant possibilities of this approach to enhance precision and efficiency in solving complex issues, quantum-inspired methods can potentially lead to significant advances in robotic technologies.

The present study focuses on the computational complexity and precision issues involved in solving the inverse kinematics of a six-degree-of-freedom industrial robotic arm, specifically the ABB IRB140. Improvements are required to allow classic methods and classical ANNs to achieve accurate and efficient solutions to this type of problem, particularly when handling high-dimensional data and potential singularity issues. These challenges necessitate innovative approaches to improve the precision and efficiency of inverse kinematics solutions. Accordingly, this study explores whether QNNs, leveraging the principles of quantum computing, can offer better performance than classical ANNs when solving inverse kinematics problems.

Several innovations introduced in the current research improve the theoretical framework, modelling techniques, and evaluation methods for solving the inverse kinematics of robotic arms using QNNs. First, an advanced kinematic model of the studied robot arm is developed, and a tailored MLP structure is designed based on the specific challenges involved in inverse kinematics. The neural network's hyperparameters are then optimised, including the number of layers, nodes, and learning rates using advanced techniques such as Gaussian noise addition and cyclical learning rates. This modelling process ensures that the neural networks can accurately predict the joint angles required for precise end-effector positioning and orientation while simultaneously avoiding singularities. Second, this study's theoretical innovation lies in integrating quantum computing principles – specifically a novel quantum-inspired activation function – into the chosen neural network's architecture. This approach employs quantum mechanics concepts such as superposition and entanglement, which are incorporated to improve the neural network's computational efficiency and learning capabilities, thereby addressing the limitations of classical methods. Finally, the advantages of our proposed approach are demonstrated by the superior

performance metrics achieved by the QNNs compared to classical ANNs. The QNNs show lower mean absolute error (MAE) values in the training and validation phases, enhanced generalisation capabilities, and enhanced singularity robustness. These benefits highlight this study's practical significance, showing that integrating quantum-inspired methods into neural networks can lead to more efficient and accurate solutions for complex robotic kinematics, with potential applications across multiple industries, including manufacturing, healthcare, and space exploration.

This paper presents a comprehensive study of using QNNs to solve the inverse kinematics of robotic arms. Section 2 describes the ABB IRB140 robot's kinematic modelling, focusing on the Denavit–Hartenberg method and Jacobian matrix. Additionally, we introduce the design and implementation of various MLP neural networks for inverse kinematics in this section, highlighting the importance of optimal network structure and training datasets. An activation function for QNNs is also proposed, and the fundamentals of quantum computing are introduced. Section 3 presents the results of our experiments and demonstrates the superior performance of QNNs in predicting the inverse kinematics of the studied robot arm. Finally, Section 4 discusses the implications of our findings and proposes directions for future research in robotics and quantum computing.

2. Methodology

In this study, we aim to develop a reliable model of the kinematics of the ABB IRB140 robot and introduce an advanced ANN architecture and a proposed QNN with a quantum-inspired activation function. The ANN is designed as a robust network to handle the general inverse kinematics of the robot arm, while the proposed quantum-inspired activation function is chosen to form a QNN that introduces non-linearity, enabling the neural network to learn complex patterns more effectively and generalise from training data to new, unseen data. Our proposed QNN model also includes mechanisms for singularity avoidance, ensuring robust and reliable performance even in challenging kinematic scenarios. Finally, the outcomes of different networks are compared, and the impacts of incorporating quantum computing on the performance of ANNs are evaluated by solving the same inverse kinematic problem using both approaches.

2.1. Kinematic modelling of the robot

The robotic arm used in this study is the ABB IRB140, a six-axis articulated manipulator known for its exceptional flexibility and extensive working range. The joint limits of this robot are listed in Table 1. The IRB140 arm is controlled by an IRC5 controller unit and is supported by PC-based RobotStudio and RobotWare software for the simulation environment [36,37].

In forward kinematics modelling, the position and orientation of the robot's end-effector are determined relative to the base frame as a function of the joint variables, which are the angles of the ABB IRB140 robot's joints. In this approach, a global coordinate frame is assigned to the robot's base, while a local reference frame is assigned to each joint.

The Denavit–Hartenberg (D-H) method is a popular approach for deriving forward kinematics models. Introduced by Jacques Denavit and

Richard Hartenberg in 1955, this convention systematically describes and represents the spatial geometry of a kinematic chain's elements, such as a robot, based on a fixed reference system [36,37]. The outcome is a 4×4 homogeneous transformation matrix that links the spatial location of the robot's end-effector to the base frame, describing both the position and orientation of the robot. Fig. 1 illustrates how the D-H parameters and links are allocated for a joint with rotational capability [36,37]. The D-H formalism is based on four parameters:

- Link length a_{i-1} : the distance between two successive joint axes z_i and z_{i-1} , measured along their common normal.
- Link twist α_{i-1} : the angle between the joint axes z_i and z_{i-1} , measured about the x_{i-1} axis.
- Link offset d_i : The distance between the x_i and x_{i-1} axes, measured along the z_i axis.
- Joint angle θ_i : The angle between the x_i and x_{i-1} axes, measured about the z_i axis.

These parameters define the homogeneous transformation matrix T_{i-1}^i between two successive frames. Consequently, the kinematic model T_0^n for a robotic arm with n joints is obtained by the product of these transformation matrices $\prod_{i=1}^n T_{i-1}^i$.

$$T_{i-1}^i = \begin{bmatrix} R & P \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} C\theta_i & -C\alpha_i S\theta_i & S\alpha_i S\theta_i & a_i C\theta_i \\ S\theta_i & C\alpha_i C\theta_i & -S\alpha_i C\theta_i & a_i S\theta_i \\ 0 & S\alpha_i & C\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1)$$

Fig. 2 illustrates the various frames and parameters associated with each joint of the studied robotic arm. Table 2 summarises the parameters for the ABB IRB 140 robot, as defined by the D-H formalism [36,37].

$${}^0T = {}^0T_1 \cdot {}^1T_2 \cdot {}^2T_3 \cdot {}^3T_4 \cdot {}^4T_5 \cdot {}^5T_6 = \begin{pmatrix} r_{11} & r_{12} & r_{13} & X \\ r_{21} & r_{22} & r_{23} & Y \\ r_{31} & r_{32} & r_{33} & Z \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (2)$$

Thus:

$$r_{11} = -S_6(S_4C_1C_{2'3} + C_4S_1) - C_6(C_5(S_1S_4 - C_4C_1C_{2'3}) + S_5C_1C_{2'3})$$

$$r_{12} = S_6(C_5(S_1S_4 - C_4C_1C_{2'3}) + S_5C_1C_{2'3}) - C_6(S_4C_1C_{2'3} + C_4S_1)$$

$$r_{13} = C_5C_1S_{2'3} - S_5(S_1S_4 - C_4C_1C_{2'3})$$

$$r_{21} = C_6(C_5(C_4S_1C_{2'3} + C_1S_4) - S_5S_1S_{2'3}) - S_6(S_4S_1C_{2'3} - C_1C_4)$$

$$r_{22} = -S_6(C_5(C_4S_1C_{2'3} + C_1S_4) - S_5S_1S_{2'3}) - C_6(S_4S_1C_{2'3} - C_1C_4)$$

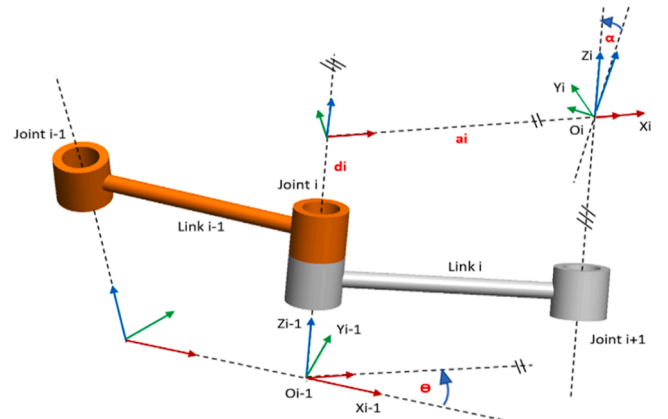


Fig. 1. D-H parameters and link assignments for a rotational joint [36].

Table 1
Joint limits of the ABB IRB 140 robot.

Joint	Type	Limits (°)
1	Rotational	+180 to −180
2	Rotational	+110 to −90
3	Rotational	+50 to −230
4	Rotational	+200 to −200
5	Rotational	+120 to −120
6	Rotational	+400 to −400

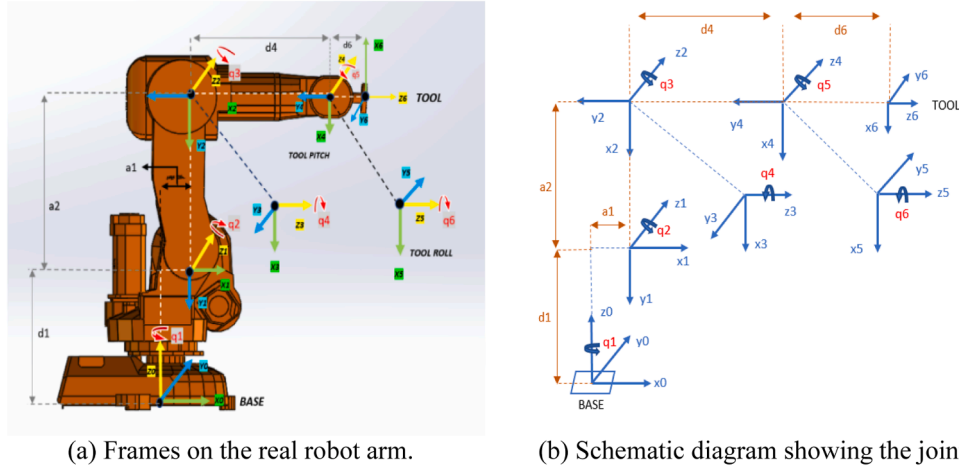


Fig. 2. ABB IRB 140 frames assignment [37].

Table 2
D-H parameters of the ABB IRB 140 arm.

Link	a (mm)	α (°)	d (mm)	q (°)
1	a1=70	-90	d1=352	q_1
2	a2=-360	0	0	$q_2 + 90$
3	0	-90	0	q_3
4	0	90	d4=380	q_4
5	0	-90	0	q_5
6	0	0	d6=65	q_6

$$r_{23} = C_5 S_1 S_{2'3} + S_5 (C_4 S_1 C_{2'3} + C_1 S_4)$$

$$r_{31} = -C_6 (S_5 C_{2'3} + C_4 C_5 S_{2'3}) + S_4 S_6 S_{2'3}$$

$$r_{32} = C_6 S_4 S_{2'3} + S_6 (S_5 C_{2'3} + C_4 C_5 S_{2'3})$$

$$r_{33} = C_5 C_{2'3} - C_4 S_5 S_{2'3}$$

where C_i and S_i denote the cosine and sine of the joint angle q_i ; C_{ij} and S_{ij} denote the cosine and sine of $q_i + q_j$; and the notation $2'$ refers to $q_2' = q_2 + \pi/2$.

The X, Y, and Z position coordinates of the IRB140 robot relative to the base frame are then computed as follows:

$$X = C_1 a_1 + d_6 (C_5 C_1 S_{2'3} - S_5 (S_1 S_4 - C_4 C_1 C_{2'3})) + d_4 C_1 S_{2'3} - C_1 C_2 a_2$$

$$Y = S_1 a_1 + d_6 (C_5 S_1 S_{2'3} - S_5 (C_4 S_1 C_{2'3} + C_1 S_4)) + d_4 S_1 S_{2'3} - C_2 S_1 a_2$$

$$Z = d_1 + d_4 C_{2'3} + a_2 S_2 + d_6 (C_5 C_{2'3} - C_4 S_5 S_{2'3})$$

The Jacobian matrix is another key component in describing the mechanism of a robotic arm. It establishes a relationship between the velocities of the end-effector (both rotational and translational) and the joint velocities, expressed as:

$$\dot{x} = J \dot{q} \quad (3)$$

The Jacobian matrix can be rotated into two sub-matrices: J_p for translational velocity and J_o for rotational velocity. Each sub-matrix contains several columns equal to the number of joints in the robot. For revolute joints, the columns of these matrices are determined by the following formula:

$$\begin{bmatrix} J_{p_i} \\ J_{o_i} \end{bmatrix} = \begin{bmatrix} Z_{i-1} \times (P_e - P_{i-1}) \\ Z_{i-1} \end{bmatrix} \quad (4)$$

Where P_e is the position vector of the end-effector, obtained from the first three elements of the fourth column of the end-effector transformation matrix T_0^e ; P_{i-1} is derived from the first three elements of the fourth column of the matrix T_{i-1}^0 ; and Z_{i-1} is taken from the third column of the rotation matrix R_{i-1}^0 . The rotation matrix R_{i-1}^0 represents the orientation of the $(i-1)$ -th link frame with respect to the base frame. It forms part of the homogeneous transformation matrix T_{i-1}^0 and is used to transform vectors from the local coordinate frame of the $(i-1)$ -th link to that of the base frame [37].

The Jacobian matrix provides a comprehensive description of the mechanism's state; however, it may become singular at specific configurations, commonly called singularity points. These points should be avoided as they can result in excessively high joint velocities and a loss of degrees of freedom within the mechanism. These singularities occur when:

$$\det J(q) = 0 \quad (5)$$

The end-effector's position description is straightforward, as it simply specifies the effector's three Cartesian coordinates in Euclidean space as defined by the X, Y, and Z axes. However, representing the orientation of the end-effector is more complex due to the multiple possible ways of representing orientation. Defining orientation requires a minimum of three variables. A typical representation is the rotation matrix, which comprehensively describes an object's orientation; however, this matrix contains nine elements, of which only three are independent, thus leading to six redundant elements. An alternative form for expressing orientation is via quaternions. Quaternions are a four-dimensional extension of complex numbers consisting of one real part and three imaginary parts. They provide a compact and efficient representation of rotations in three-dimensional space [23,25].

The closed-form quaternion method for extracting a quaternion from a rotation matrix, based on the Sarabandi and Thomas method, begins with Euler's Theorem of Rotations. Euler's theorem states that any rotation in three-dimensional space can be represented as a single rotation about a specific axis, which can be formulated as a rotation through an angle θ around some unit vector $\tilde{n} = (n_x, n_y, n_z)^T$ [25,38]. The corresponding rotation matrix $R(\tilde{n}, \theta)$ is given by:

$$R(\tilde{n}, \theta) = \begin{bmatrix} c + n_x^2(1-c) & n_z n_y(1-c) + n_x s & n_x n_z(1-c) - n_y s \\ n_x n_y(1-c) - n_x s & c + n_y^2(1-c) & n_y n_z(1-c) + n_x s \\ n_x n_z(1-c) + n_y s & n_y n_z(1-c) - n_x s & c + n_z^2(1-c) \end{bmatrix} \quad (6)$$

Where c and s represent the cosine and sine functions of an angle.

By introducing a non-zero real number k and quaternion components q_0, q_1, q_2 , and q_3 , we can express the rotation matrix in terms of quaternion components:

$$R(q) = \frac{1}{q_0^2 + q_1^2 + q_2^2 + q_3^2} \begin{bmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1q_2 + q_0q_3) & 2(q_1q_3 - q_0q_2) \\ 2(q_1q_2 - q_0q_3) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2q_3 + q_0q_1) \\ 2(q_1q_3 + q_0q_2) & 2(q_2q_3 - q_0q_1) & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{bmatrix} \quad (7)$$

For a rotation matrix $R = (r_{ij})_{1 \leq i, j \leq 3}$, a 4×4 symmetric matrix U can be defined:

$$U = \frac{1}{4} \begin{bmatrix} r_{11} + r_{22} + r_{33} + 1 & r_{32} - r_{23} & r_{13} - r_{31} & r_{21} - r_{12} \\ r_{32} - r_{23} & r_{11} - r_{22} - r_{33} + 1 & r_{21} + r_{12} & r_{31} + r_{13} \\ r_{13} - r_{31} & r_{21} + r_{12} & r_{22} - r_{11} - r_{33} + 1 & r_{32} + r_{23} \\ r_{21} - r_{12} & r_{31} + r_{13} & r_{32} + r_{23} & r_{33} - r_{11} - r_{22} + 1 \end{bmatrix} \quad (8)$$

The dominant eigenvector of U corresponds to the quaternion representing the same rotation as R . The elements r_{11}, r_{22} , and r_{33} are the diagonal elements of the rotation matrix R , and the elements $r_{32}, r_{23}, r_{13}, r_{31}, r_{21}$, and r_{12} are the off-diagonal elements. We use these elements in constructing the symmetric matrix U , which is related to the quaternion representation of the rotation. We can estimate the quaternion $\tilde{q}$ from the rotation matrix by averaging the column vectors of U with homogenised signs:

$$\tilde{q} = \sum_{i=1}^4 \text{sign}(u_{ij} \cdot u_i) u_i \quad (9)$$

where u_j is chosen such that $u_j^2 \geq u_i^2$ for $i = 1, \dots, 4$. This method ensures that the averaged quaternion $\tilde{q}$ represents a rotation matrix, irrespective of the sign of $\det(R)$.

2.2. Multi-layered perceptron (MLP)

An MLP neural network is a versatile ANN structure widely used in robotics for control, path planning, and object recognition tasks. Unknown or computationally intensive algorithms cause challenges when solved using classic algorithmic approaches in complex systems. Neural networks, which can be used for both regression and classification tasks, represent a potential solution to these issues by reducing software development time and computational requirements and allowing processing when algorithms are undefined.

Our approach integrates a quantum-inspired activation function into the MLP neural network. This function leverages quantum principles to introduce non-linearity and enhance the network's ability to learn complex patterns. By encoding high-dimensional input data into quantum states during the initial state preparation and operation transformation steps, our approach enables the neural network to more efficiently process information. This method not only accelerates the learning process but also enhances the network's ability to generalise from training data to new, unseen data. We also further incorporate singularity avoidance mechanisms to ensure the robustness of our proposed solution in practical applications.

In robot kinematics, predicting the inverse kinematics of robotic arms represents a regression problem, where the input is the position and orientation of the end-effector, and the output is the joint angles. In this regression task, the purpose is to predict continuous output values

(i.e. the joint angles) [39,40]. The neural network model for the regression process requires a dataset with appropriate inputs and outputs, as shown in Fig. 3. The input should be a vector describing the end-effector's position and orientation, while the output should be the corresponding joint angles.

There are two key challenges involved in exploring the inverse kinematics of robotic manipulators with ANNs: choosing an optimal neural network structure and generating a robust training dataset [41, 42]. This research focuses on various MLP structures to maximise the accuracy of approximations.

To obtain the most reliable and realistic data for training the network, we used the studied robotic arm to execute tasks and generate real-world data. This approach captures the complexity and variability of real-world physical systems, thus providing more robust data for model training than can be obtained via simulation. While simulation-based or theoretical methods are common in robotics research due to their convenience and cost-effectiveness, these approaches do not offer the same level of reliability as data derived from actual tasks performed using the studied robot. By integrating real-world data and advanced preprocessing steps, our approach ensures that the neural network is exposed to a wide range of scenarios and dynamics representative of operational conditions. This strategy enhances the robustness and generalisation of the trained models, ensuring higher precision and adaptability in practical scenarios [V2].

The robot underwent path planning and executed various trajectories to gather the necessary data to create the ANN dataset. The resulting dataset consists of 13 elements: three for position vectors and four for orientation (representing a total of seven input elements), and six for joint angles (as outputs). We operated the IRB 140 ABB Robot

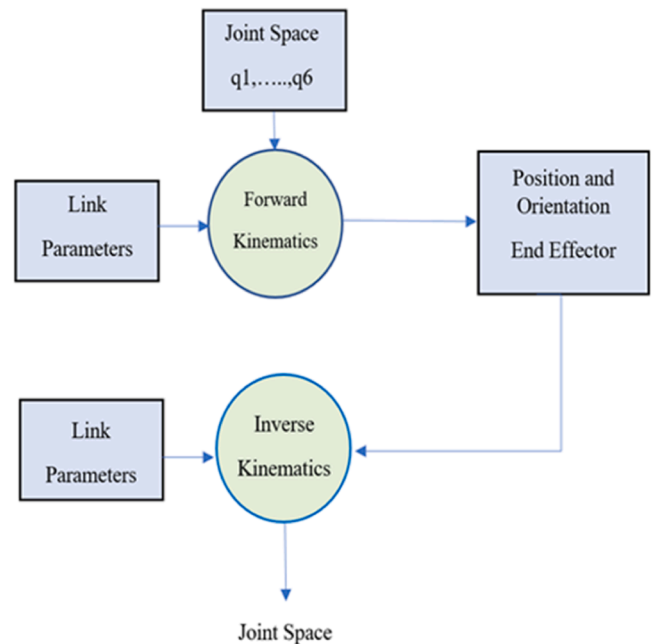


Fig. 3. Kinematics data flow diagram of the robotic manipulator arm.

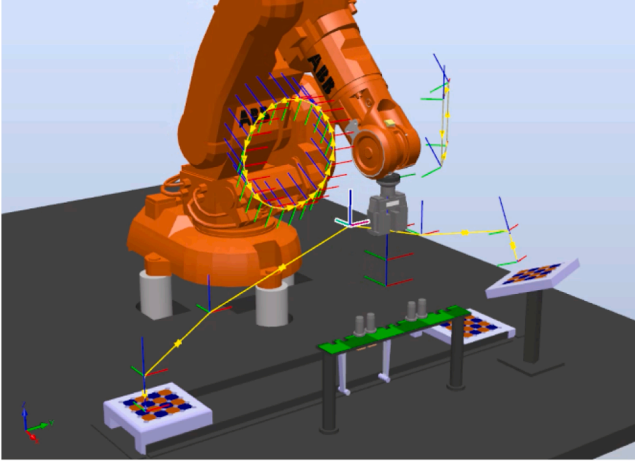


Fig. 4. IRB 140 ABB robot manipulator arm during executing assigned tasks.

manipulator arm in the mechatronic laboratory of the University of Quebec in Trois Rivières to generate approximately 167,000 data points, covering each joint's full range of motion while executing various tasks, as presented in Fig. 4 (a screenshot of the IRC5 interface). The ABB IRB 140 is a high-degree-of-freedom robot with the ability to perform various types of challenging motions rapidly and accurately. This robot arm has ample working space and can be applied to numerous industrial scenarios, ensuring the relevance and applicability of our findings across a broad range of practical applications. The dataset we generated is extensive and covers the entire working space of the ABB IRB 140 robot, thus ensuring our neural network models are trained on varied and representative sets of movements and joint configurations, improving the robustness of the study's outcomes. The selected trajectories included triangular, square, rectangular, and elliptical paths, symmetric and asymmetric polygons of different sizes, and random paths within the robot's working space in the variant scale range for each defined path. These trajectories simulate practical tasks such as welding, pick-and-place operations, painting, and assembling, thus ensuring the obtained data apply to real-world scenarios and capture a wide range of motion dynamics.

We applied a two-step normalisation process to the dataset to ensure practical neural network training and prevent divergence, which involved normalising the range to -1 and $+1$ and applying a Gaussian signal for a normal distribution. Additionally, we added Gaussian noise with a standard deviation of 0.002 to the data.

These MLPs determine the joint angles that produce the desired end-effector positions and orientations. The network's architecture is shown in Fig. 5. The initial input layer of the MLP model ranges from three to seven inputs, depending on whether it includes both position and orientation. Several hidden layers follow, with the output from the final hidden layer comprising six elements representing the joint angles. The network includes a customised layer that ensures these angles remain

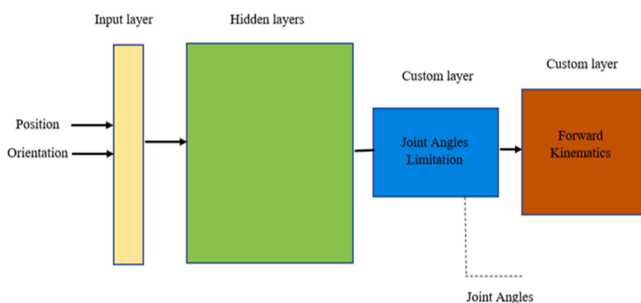


Fig. 5. Neural network architecture.

within the robot's operational range and an additional customised layer applies the direct kinematics model outlined in Section 3.1. Both customised layers were developed in MATLAB software and implemented in the robot's network architecture, ensuring that the MLP's outputs are suitable for motion planning. The architecture includes performance-measuring loss functions and MAE measurements, which quantify the deviation between the estimated and actual end-effector poses and can be expressed as:

$$MAE = \frac{1}{n} \sum_{i=1}^n |R_i - \hat{R}_i| \quad (10)$$

Where R_i represents a sample from the dataset, and $\hat{R}_i$ is the predicted value for this sample.

A comprehensive hyperparameter search can enhance the designed network's performance by optimising the learning rate, batch size, network architecture (depth and width), and choice of non-linear activation functions. This process also examines the number of training epochs and data pre-processing techniques, such as Gaussian noise addition or batch normalisation. The ANN architecture proposed in this study is shown in Fig. 5.

The hyperparameter tuning process applied in our study is stationary instead of time-dependent, meaning that the hyperparameters are selected and evaluated independent of any specific temporal events or fixed periods. Instead, the tuning is based on the network's overall performance across the entire dataset, thus ensuring consistent and comprehensive evaluation. The random search for hyperparameters was conducted with the dataset split into training, validation, and testing sets. This splitting ensured a comprehensive evaluation of the model's performance and prevented any temporal biases. During each iteration of the random search, hyperparameters such as the number of nodes, layers, and learning rate were randomly selected within predefined ranges. The model was then trained and evaluated based on the mean squared error (MSE) loss. The first round of random searching provided an initial set of hyperparameter values. A second round of tuning was then performed, refining the search within narrower ranges based on insights from the first round. The hyperparameters were assessed by evaluating the model's performance on the validation set after each training session, thus ensuring that this assessment was based on a broad and consistent data sample, preferably varying over time.

In the hyperparameter tuning process, numbers of nodes from 5 to 535 and layers from 3 to 53 were tested, with a logarithmic scale learning rate ranging between $1e-5$ and $8.8e-4$. The Adam optimiser was used in all experiments due to its balanced approach to gradient scales and rapid convergence, thus making it suitable for handling complicated tasks. A random search was conducted on a dataset with 167,000 data points (70 % training, 15 % validation, and 15 % testing), with a batch size of 32 and 170 epochs of training to determine appropriate values for the numbers of layers and nodes and the learning rate range. The ReLU activation function and MSE loss function were employed. Although the initial experiments provided valuable insights, learning rate values outside the initial search range yielded better results, prompting a second round of random searching.

After two rounds of hyperparameter tuning, we discovered that models with 3–15 layers produced the most accurate results, particularly those with four layers. Node counts around 488 within the approximate range of 450–530 also displayed a lower loss. A learning rate below $5e-5$, specifically $1.9e-5$, was found to be the most efficient. Although the initial experiments indicated effective ranges for the learning rate, further tuning outside these initial ranges (specifically identifying a learning rate of $1.9e-5$) was necessary. These adjustments were based on the observed performance metrics rather than time-dependent factors. Fig. 6 summarises these findings, illustrating the complex relationship between network architecture and performance [v2].

Non-linear activation functions were assessed using the identified optimal MLP architecture in terms of the numbers of nodes and layers,

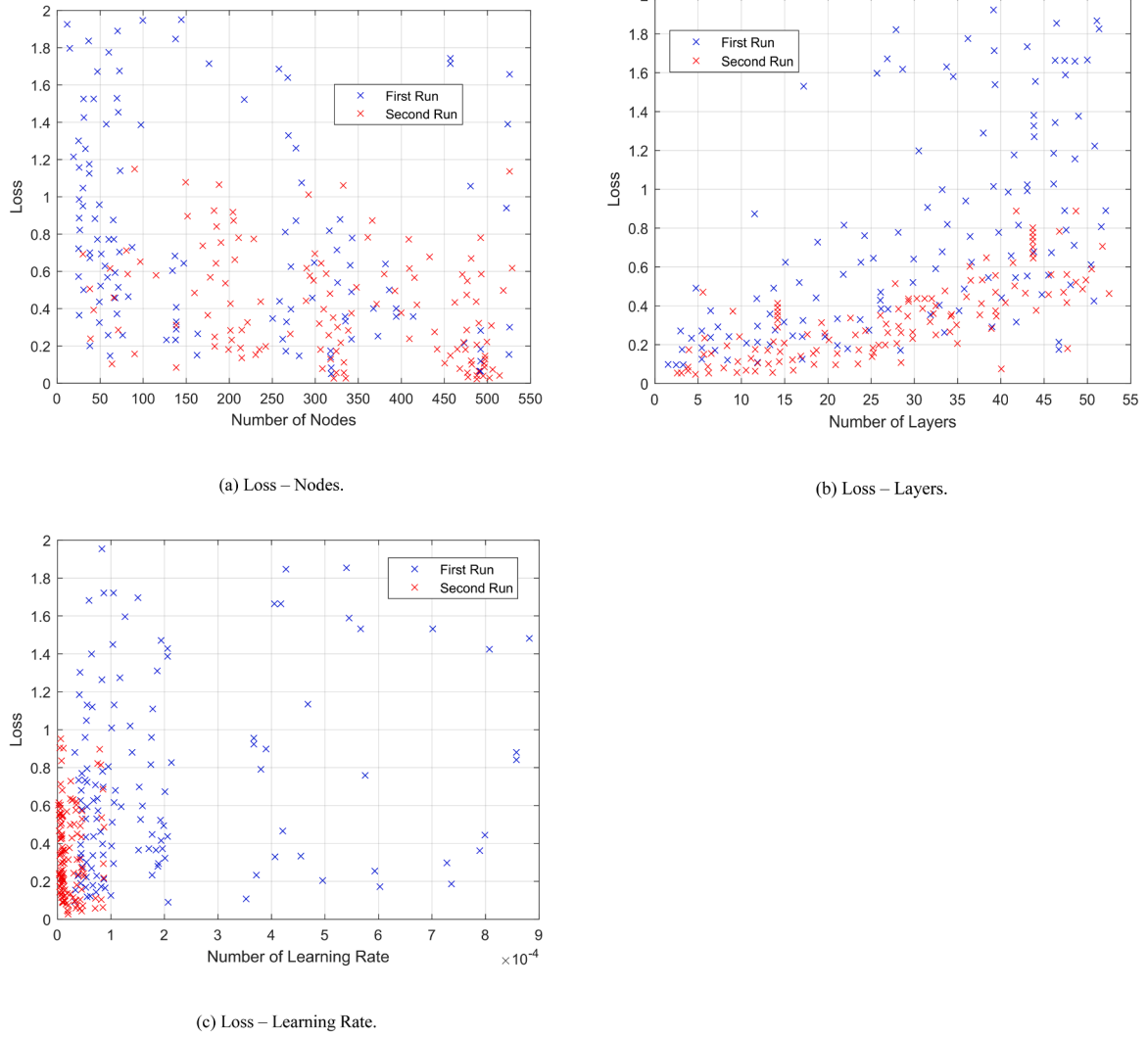


Fig. 6. Plots showing the loss values for varying (a) nodes, (b) layers, and (c) learning rates for the proposed ANN.

Table 3

Performance comparison of the various studied activation functions.

Function	ReLU	Swish	Hardlims	Satlins	Radbas
Epoch	287	300	83	47	221
Time	00:52	00:14	01:00	00:33	01:14
Performance	0.305	0.306	32.4	0.694	0.294
Gradient	0.168	0.324	3.30e-11	0.0735	0.214
Regression	0.99981	0.99981	0.97946	0.99957	0.99979
Validation Check	0	0	4	12	12
Max Error	0.08051	0.02943	1.069	0.03508	0.09044

using batch sizes of 32 and 64. The Swish activation function was found to be the most effective, achieving zero validation error, the smallest maximum error among the studied functions of 0.02943, and an almost perfect regression value of 0.99981, indicating strong alignment between the predicted and actual outcomes, with a performance metric value of 0.306. The Swish function also displayed stability during training, as evidenced by the absence of validation check failures, suggesting no overfitting occurred. A comparison between the studied activation functions is shown in Table 3, highlighting the significance of selecting appropriate functions to ensure optimal MLP performance.

2.3. Singularity points avoidance

Robotic manipulator arms can encounter two types of singularities: boundary and interior singularities. Boundary singularities occur at the edges of the workspace and are related to the physical limits of the robot's joints. In contrast, interior singularities occur within the workspace due to the alignment of robot axes, leading to a loss of degrees of freedom [43]. Motion can become problematic when singularities arise, causing joint velocity and torque issues. The Jacobian matrix is crucial for velocity kinematics as it maps joint velocities to end-effector velocities [44]; however, it becomes non-invertible at singularities, where the determinant of the Jacobian matrix is equal to zero. The condition number, denoted by k , is a measure of a matrix's sensitivity to small changes in its input values [45]. This parameter ranges from 1 to infinity, with higher values indicating an ill-conditioned matrix, in which the mechanism is in a singularity point region, making it challenging to find accurate solutions. Lower condition numbers imply that the mechanism is far from singularity points, and small changes in input values will not significantly affect the solution. The condition number of the matrix is defined as:

$$\kappa(A) = A^{-1} \|A\| \quad (11)$$

Close to singularities, even small velocity inputs can produce large outputs due to a high condition number, despite the Jacobian

maintaining full rank. For redundant manipulators, which have more degrees of freedom than necessary for a given task, the Jacobian matrix is non-square. Such cases use a pseudo-inverse of the Jacobian, achieved through singular value decomposition [46]. To address the challenges of singularities, we can formulate a loss function for the manipulator. This loss function combines the MAE between the target position (P_{Target}) and the calculated position (P_{Calc}), with a penalty for high Jacobian condition numbers above a certain threshold. We can express the loss function as:

$$\text{Loss Function} = \text{MAE}(P_{\text{Target}}, P_{\text{Calc}}) + f(\text{Activation Function}, \kappa(\text{Jacobian})) \quad (12)$$

The singularity avoidance neural network takes the end-effector's desired position as an input. It disregards its desired orientation, thus corresponding to a mechanism with three degrees of freedom that can avoid singularity configurations based on the Jacobian matrix's condition number. The loss function utilises MAE and the Jacobian. By incorporating this penalty, the manipulator is encouraged to avoid configurations close to singularities, thus improving its performance and reliability.

2.4. Evaluation of the ANN

Metrics can be used to assess the performance of neural network models by quantifying the error in the end-effector's position and orientation relative to the desired target. The Euclidean distance between the calculated and target positions of the end-effector is given by:

$$d = \sqrt{(X - X')^2 + (Y - Y')^2 + (Z - Z')^2} \quad (13)$$

where $P = [X, Y, Z]$ and $P' = [X', Y', Z']$ are the vectors of the calculated and target positions, respectively.

We can use several metrics to calculate the orientation error. Evaluating the orientation error between two rotation matrices, R_1 and R_2 , is crucial in neural network models for robotics, especially in tasks involving the control of robotic arms. Rotation matrices are commonly used to assess this error, or the quaternion representation for rotations can be used as an alternative to Euler angles [47]. Quaternion-based metrics for the orientation error include the minimum quaternion error, which is calculated as:

$$\phi(q_1, q_2) = \min(q_1 - q_2, q_1 + q_2) \quad (14)$$

This metric determines the error as the minimum norm of the difference between the calculated quaternion q_1 and the target quaternion q_2 . Another quaternion metric is the arccosine of the absolute dot product of q_1 and q_2 , i.e.

$$\phi(q_1, q_2) = \arccos(|\langle q_1, q_2 \rangle|) \quad (15)$$

This metric measures the error as an angle within $[0, \frac{\pi}{2}]$. Additionally, the magnitude of the quaternion difference quantifies the difference between one quaternion and another within the range $[0, 1]$, which is measured as:

$$\phi(q_1, q_2) = 1 - |\langle q_1, q_2 \rangle| \quad (16)$$

Orientation error can also be defined using rotation matrix-based metrics. In these metrics, the error is quantified based on the distance between the two matrices. For instance, the Frobenius norm of the difference between the rotation matrices R_1 and R_2 is given by:

$$\delta(R_1, R_2) = \sqrt{2(3 - \text{trace}(R_1 R_2^T))} \quad (17)$$

The logarithm of the rotation matrix difference is another valuable metric as it represents the maximum angle of separation of points transformed by R_1 and R_2 and is expressed as:

$$\delta(R_1, R_2) = \log(R_1 R_2^T) \quad (18)$$

Finally, the arccosine of the trace of rotation matrix difference is a metric that reflects the orientation error as an angle within $[0, \pi]$ and is given by:

$$\delta(R_1, R_2) = \arccos\left(\frac{\text{trace}(R_1 R_2^T) - 1}{2}\right) \quad (19)$$

Selecting an orientation error metric should be based on the specific application's needs, such as available computational resources, the nature of the rotations, and the desired sensitivity to small errors. The logarithm of rotation matrix difference metric is often preferred as it intuitively represents the error as an angle, consistent with the geometric interpretation of rotation in three-dimensional space.

2.5. Quantum computing basis

Quantum computing marks a considerable departure from classical computing. By using the principles of quantum mechanics, quantum computing processes information fundamentally differently: whereas classical computing relies on binary bits of either 0 or 1, whereas quantum computing utilises the qubit or quantum bit. Through the two fundamental quantum mechanical phenomena of superposition and entanglement, qubits can represent a state of 0, 1, or any quantum superposition of the two [48–50]. Superposition allows a qubit to exist simultaneously in a combination of 0 and 1 states. The state of a qubit, $|\psi\rangle$, can be expressed as a linear combination of the basis states $|0\rangle$ and $|1\rangle$ [23,49,51]:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \quad (20)$$

Here, α and β are complex numbers representing the probability amplitudes of the qubit's state. Upon measurement, the square of the amplitude's modulus, $|\alpha|^2$, gives the probability of the qubit being in state $|0\rangle$, and similarly, $|\beta|^2$ describes the probability of it being in state $|1\rangle$. For a qubit, these probabilities must sum to one, satisfying the normalisation condition [24,52].

A qubit's state can be visualised on the Bloch sphere, where $|0\rangle$ and $|1\rangle$ are depicted as orthogonal vectors. An arbitrary qubit state on the Bloch sphere using two angles, θ and ϕ [24,53,54]:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle \quad (21)$$

In this representation, θ represents the latitude from the sphere's north pole, while ϕ is the longitudinal angle. The qubits are manipulated through quantum gates, which are represented by unitary operators. These gates are the quantum counterparts of classical logic gates and operate reversibly. Rotations about the axes of the Bloch sphere are essential quantum operations, allowing for the transformation of qubit states through rotation operators such as [25,54,55]:

$$R_x(\lambda) = \begin{pmatrix} \cos\left(\frac{\lambda}{2}\right) & -i\sin\left(\frac{\lambda}{2}\right) \\ -i\sin\left(\frac{\lambda}{2}\right) & \cos\left(\frac{\lambda}{2}\right) \end{pmatrix} \quad (22.a)$$

$$R_y(\theta) = \begin{pmatrix} \cos\left(\frac{\theta}{2}\right) & -\sin\left(\frac{\theta}{2}\right) \\ \sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{pmatrix} \quad (22.b)$$

$$R_z(\phi) = \begin{pmatrix} e^{-i\phi/2} & 0 \\ 0 & e^{i\phi/2} \end{pmatrix} \quad (22.c)$$

where λ , θ , and ϕ are the rotation angles around the x, y, and z axes, respectively.

Entanglement is a vital quantum effect in which qubits become corresponding, making their states linked, even across extensive spaces [56,57]. This property is essential in achieving the intricate multi-qubit states of quantum computing. A key aspect of entanglement is the rotation around the y-axis of the Bloch sphere, making it possible to generate the necessary superposition and entangled states required for operations such as error modification and teleportation [58,59]. For instance, applying $R_y(\theta)$ to a qubit initially in state $|0\rangle$ can result in a superposition essential for algorithms like Grover's search algorithm or the quantum Fourier transform. This operator is defined in Eq. (22.b) [60,61].

The R_y gate holds significant power in creating entangled and superposition states. By manipulating the rotation angle θ , complex systems can be simulated using quantum mechanics, allowing the emulation of quantum dynamics relevant to fields like material science and quantum chemistry. Incorporating quantum gates, including rotations, enables the creation of intricate quantum circuits with computational capabilities exceeding those of classic computing. Specifically, rotations around the Y-axis are leading the advancement of quantum algorithm development and unlocking the full potential of quantum computing.

2.6. Proposed quantum activation function

Here, we propose a quantum activation function, with a focus on the initial state preparation and the operation transformation operator. The initial state preparation is fundamental in encoding high-dimensional input data [62], such as elements of robotic motion, into quantum states by generating superposition states and encoding the input data into the phase of the quantum state. However, when applied correctly, the transformation operator can introduce non-linearity to the input data, which is essential for effectively implementing the quantum neural network's activation function [63].

The initial state preparation step represents preparing a qubit in a superposition state, generally using a quantum gate for equal superposition [64]. After a gate is used to put each qubit into a superposition state, we then apply phase rotation gates, which encode the input data $\tilde{x}_j$ into the phase of the quantum state mathematically formulated as:

$$|\psi(x)\rangle = \prod_{j=1}^N U(\tilde{x}_j) |0\rangle \quad (23)$$

Where $U(\tilde{x}_j)$ represents a general quantum operation or series of operations applied to each qubit, encompassing both the creation of superposition and the encoding of input data $\tilde{x}_j$ into a quantum state.

The selection of the terms for the equation initial state preparation (24) can be formulated as the exponential of $i\sigma_y x_j$ for phase rotation and $\text{Sqrt}(Y)_j$ for the square root of the Y gate. We select these elements for their proficiency in encoding high-dimensional input data into quantum states, allowing an effective description of robotic movements. Eq. (24) describes the initial state preparation of a quantum system – here, the input data are represented by x_j , where $\text{Sqrt}(Y)_j$ is applied to the j -th qubit, replacing the Pauli Y gate for phase rotation. The $\text{Sqrt}(Y)_j$ gate generates a superposition state from the base state $|0\rangle$, while the exponential of $i\sigma_y x_j$ induces a phase rotation. This method of state preparation is inherently non-linear relative to the input data x_j , combining phase encoding with superposition to augment the quantum model's ability to link positions and orientations with joint angles.

$$|\psi(x)\rangle = \prod_{j=1}^N e^{i\sigma_y x_j} \cdot \text{Sqrt}(Y)_j \cdot |0\rangle \quad (24)$$

The operation transformation operator is another essential part of the quantum activation function, as it manipulates the state to introduce

non-linearity relative to the input data – this can be effectively used to implement the quantum neural network's activation function by adjusting the state of each qubit based on these parameters, which can be optimised during the learning process [65]. An operation transformation as a special unitary two-dimensional complex vector represents all possible single-qubit rotations on the Bloch sphere [66]. Single-qubit operations are preferred for their ease of input data encoding and quantum function design, thus enabling a straightforward exploration of the impact of quantum approaches on neural networks without the complexities of entanglement and multi-qubit interactions.

$SU(2)$ operations (special unitary group of degree 2) are linear because of the unitary nature of all operations, thus ensuring the reversibility of the quantum operations and the preservation of the total probability [67]. While all quantum operators are linear due to the requirements of unitarity, applying these operators in quantum computing algorithms can result in non-linear functionalities due to the complex interplay of quantum states, entanglement, superposition, and measurement processes. An $SU(2)$ operation on a single qubit can be represented as a rotation in the three-dimensional Bloch sphere given by:

$$U(\theta, \phi, \lambda) = e^{-\frac{i\phi\sigma_z}{2}} e^{-\frac{i\theta\sigma_y}{2}} e^{-\frac{i\lambda\sigma_z}{2}} \quad (25)$$

Where θ is the rotation angle around the y-axis, ϕ is the initial phase rotation around the z-axis, λ is the final phase rotation around the z-axis, and σ_x , σ_y , and σ_z are the Pauli matrices corresponding to rotations around the x, y, and z axes of the Bloch sphere, respectively.

Applying the $SU(2)$ operation to the prepared qubit from the previous step can be used to explore the entire Bloch sphere. The $SU(2)$ operation represents all possible single-qubit rotations on the Bloch sphere, meaning it can be used to adjust the qubit's state in any way. This operation allows a comprehensive exploration of the quantum state space and enables the activation function to introduce non-linearity based on the input data.

$$|\tilde{\psi}(x)\rangle = \prod_{j=1}^N e^{-\frac{i\phi\sigma_z}{2}} e^{-\frac{i\theta\sigma_y}{2}} e^{-\frac{i\lambda\sigma_z}{2}} \cdot |\psi(x)\rangle \quad (26)$$

To assess the quantum neural network's performance, we applied the proposed quantum-inspired activation function instead of the Swish activation function, considering the requirements for transforming, normalising, and encoding the input and target data. We evaluated the network employing the optimal hyperparameters identified in section 3.2. We selected MATLAB as the primary computational tool for coding and managing the studied networks throughout the training and evaluation processes. In the upcoming section of the paper, we will present and analyse the results of our comparative study between classical neural networks and QNNs, thus providing valuable insights into the effectiveness of our proposed approach.

3. Results and discussion

In this study, we compared the performance of two types of neural networks with different activation functions. The first neural network utilised the Swish activation function, which demonstrated the best performance among the tested activation functions, as shown in Table 3. The second network employed an MLP architecture with a novel quantum-inspired activation function. Additionally, we assessed two other network configurations by integrating a custom layer for singularity avoidance and comparing standard MLP models with quantum-inspired MLP models.

The findings reveal that the QNNs outperform classical ANNs in predicting the inverse kinematics of the ABB IRB140 robotic arm. The QNNs showed lower MAE values during the training and validation stages, indicating better precision and improved generalisation capabilities. The introduction of a quantum-inspired activation function was key in this advancement, allowing more effective learning and

mitigating overfitting. Furthermore, the QNNs displayed improved singularity avoidance, further verifying the efficacy of the proposed approach in addressing challenging kinematic scenarios.

We established the chosen hyperparameters for our model through several iterations. We applied the Swish activation function and quantum-inspired activation function in the MLP layers and trained the model using the MAE loss function. We set the batch size to 64, and the training process spanned 170 epochs. We observed that a constant learning rate did not yield satisfactory results – high learning rates led to oscillating loss values, whereas low learning rates resulted in convergence to a saddle point rather than the global optimum. To address this problem, we implemented a cyclical learning rate strategy.

Analysing a model's training and validation errors is crucial to evaluating its performance and accuracy. While the training error measures the model's ability to learn from the training data, the validation error assesses its capacity to generalise to new, unseen data. Understanding these errors is essential for ensuring the model can reliably capture the underlying patterns in the data and make accurate predictions in real-world scenarios.

In this study, we examined the training and validation errors to develop a model that both performs well on the training dataset and exhibits robust generalisation capabilities when exposed to new data. The performance of the classical and quantum-inspired networks based on an MLP architecture is shown in Figs. 7 and 8. To enhance the model's generalisation capability, we introduced Gaussian noise with a standard deviation of 0.0035 as the initial layer of the model.

Fig. 7 shows the MAE loss of the studied ANN across 170 epochs, revealing an initial rapid learning phase, with significant decreases in the training and validation losses. Within the first 20 epochs, the training loss drops to around 0.3 and stabilises, mirrored by the validation loss, indicating promising initial generalisation. Towards the end of the training, the convergence of training and validation losses (MAE values of 0.0098825 and 0.010956, respectively) demonstrate effective learning without notable overfitting, indicating that the model will perform well when presented with new data.

Fig. 8 shows the QNN's MAE loss during training and validation. Initially, the loss drops sharply, indicating rapid learning, then stabilises, indicating convergence towards a solution. The final MAE loss values for training and validation are 0.0065353 and 0.007236, respectively, indicating the model has good generalisation ability without overfitting. Overall, these results confirm the proposed QNN's effective learning capability and potential accuracy in real-world applications.

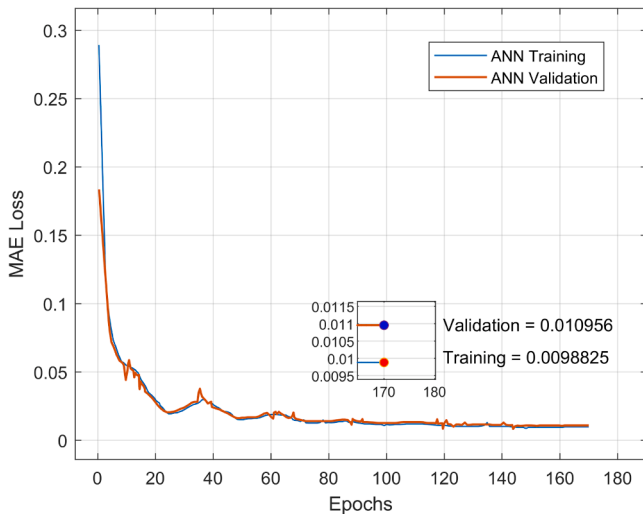


Fig. 7. Loss values of the ANN for position and orientation versus number of epochs.

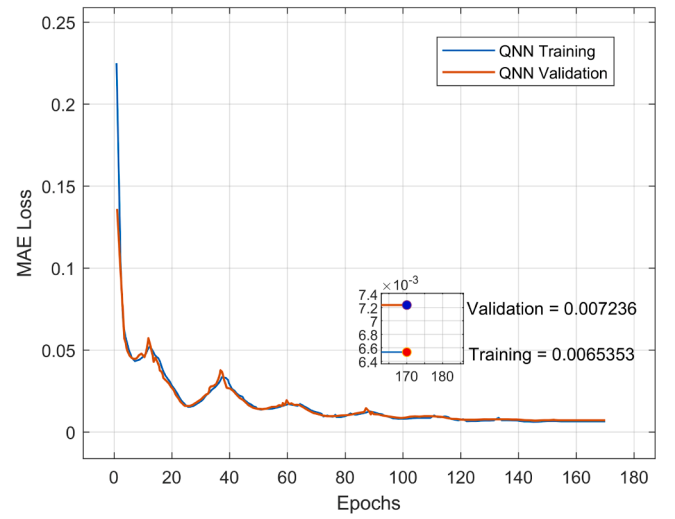


Fig. 8. Loss of QNN for position and orientation.

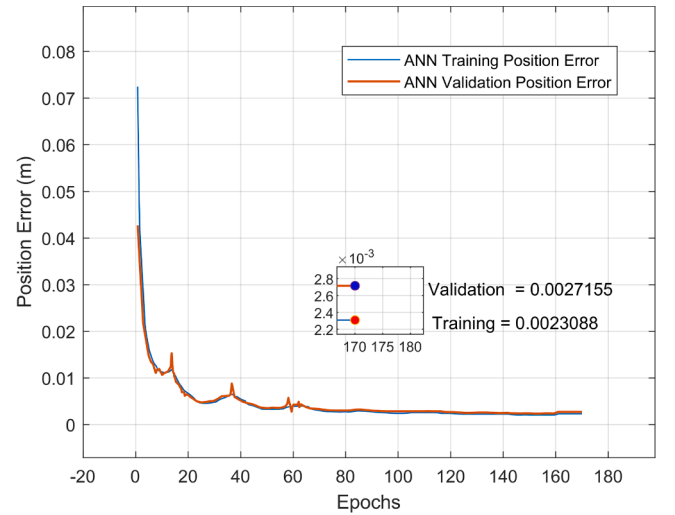


Fig. 9. Position error of ANN during training and validation.

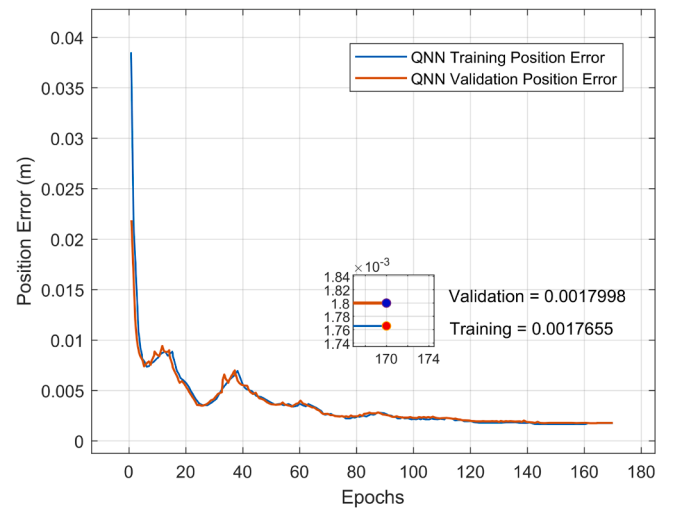


Fig. 10. Position error of QNN during training and validation.

A qualitative comparison of the performance and accuracy of the QNN and ANN indicates superior performance in the QNN, which exhibits rapid learning capabilities and stabilises at a more optimal value, indicating higher efficiency in learning and generalising from data.

Both networks lack overfitting; however, the proposed QNN appears to align more closely with unseen data, as evidenced by its lower validation MAE loss, suggesting robust generalisation. While they do not provide exact percentages, the presented statistics imply that the QNN is the more accurate and better-performing network in this comparison.

The training and validation position errors of the networks over 170 epochs are presented in Figs. 9 and 10. As shown in Fig. 9, the position error in the ANN drops rapidly, indicating rapid learning. Both errors converge towards a constant value as the number of epochs increases, showing that learning has stabilised.

The final training error is 2.3 mm, with a slightly higher validation error of 2.71 mm, indicating the ANN's ability to generalise well without overfitting. The plateauing of the position error after the initial drop suggests that the ANN has reached its learning capacity in the studied configuration.

In the results shown for the QNN in Fig. 10, the algorithm initially shows a sharp decrease in error, indicating rapid learning and adaptation. As the epochs progress, both the training and validation errors stabilise at a low level, suggesting that the QNN has achieved its optimal state of learning. After 170 epochs, the training position error settles at approximately 1.76 mm, with the validation error slightly higher at around 1.79 mm. The negligible difference between these values indicates a well-generalised model with minimal overfitting, which is crucial for real-world applications that require accurate predictions based on new, unseen data. The graph also indicates that the errors align closely and remain consistent after the initial drop, demonstrating that the current network architecture maximises the model's learning capacity. This behaviour showcases a mature model that can reliably perform tasks that require precise positional estimations, validating the effectiveness of the proposed QNN in these applications.

Overall, this comparison reveals that the QNN presents improved capabilities with a significantly lower position error, indicating its suitability for high-precision practical applications. Its consistent and stable error reduction suggests a strong learning ability within the proposed network architecture. Furthermore, with a validation position error approximately 1 mm lower than that of the ANN, the QNN displays superior proficiency in handling new and unseen data, highlighting its exceptional generalisation ability.

The orientation error for both networks during training and validation is presented in Figs. 11 and 12. For the ANN in Fig. 11, both errors

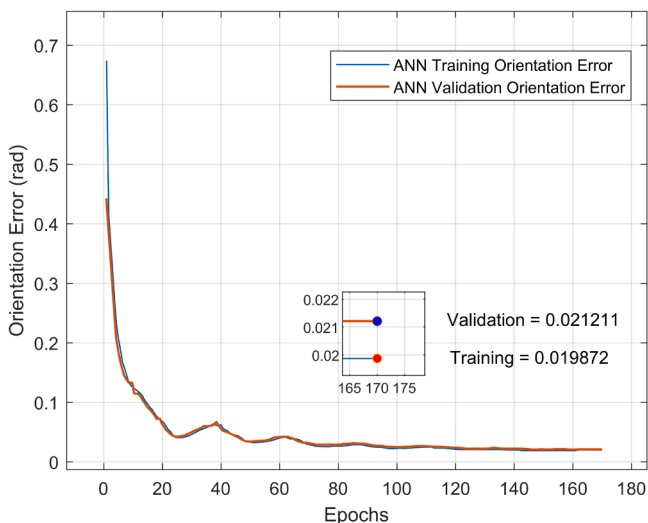


Fig. 11. Orientation error of ANN during training and validation.

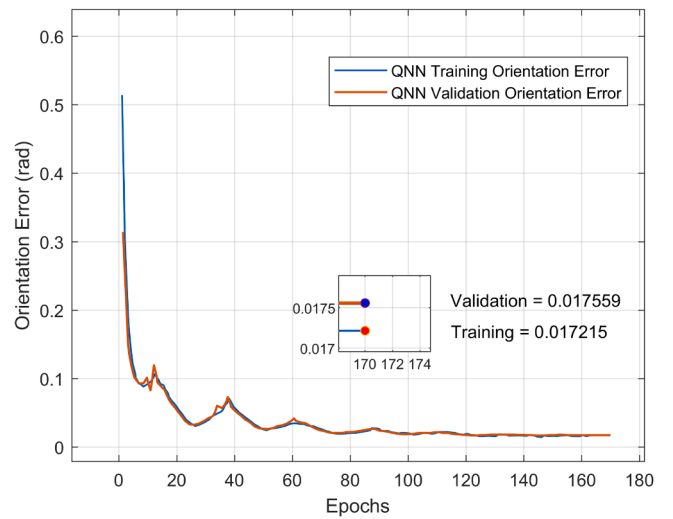


Fig. 12. Orientation error of QNN during training and validation.

gradually decrease and stabilise at a low level, with the training error slightly lower than the validation error. The inset shows a magnified view of the graph's tail, revealing that the training and validation errors are approximately 0.0198 and 0.021 radians in the 170th epochs. This slight difference between training and validation errors indicates that the model can make accurate predictions without significant overfitting. The final error values demonstrate the ANN's strong predictive capabilities, indicating that it has learnt the task with remarkable accuracy. This level of accuracy is essential for high-precision applications like robotics or complex system modelling.

The QNN network in Fig. 12 displays a sharp initial decrease in training error, indicating rapid adaptation to the training set. The training and validation errors progressively converge, with the training error slightly lower than the validation error, implying effective learning without significant overfitting. The inset highlights the tail end of the graph, showing the close similarity between the training and validation errors in the final epochs. Notably, the errors stabilise at a value lower than that of the ANN, with training and validation errors recorded of approximately 0.00172 and 0.00175 radians, respectively. The minimal discrepancy between these values indicates the robust predictive abilities of the QNN. Overall, similar to the trends observed for the position error, these results indicate that the QNN outperforms the ANN in terms of orientation error, showcasing its precise learning capabilities.

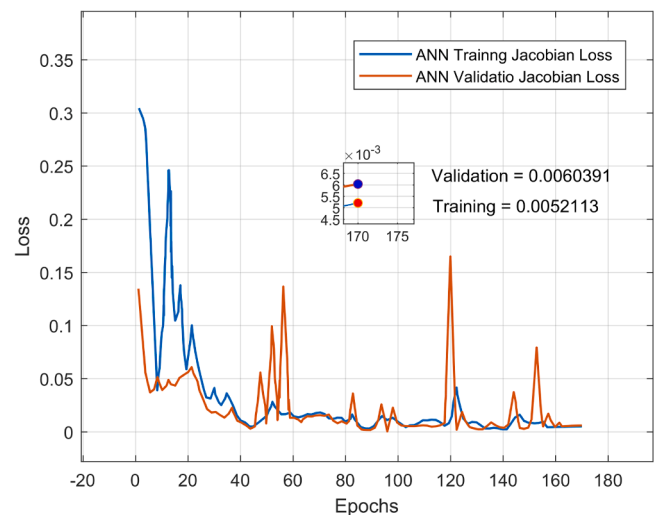


Fig. 13. Loss of ANN for singularity avoidance.

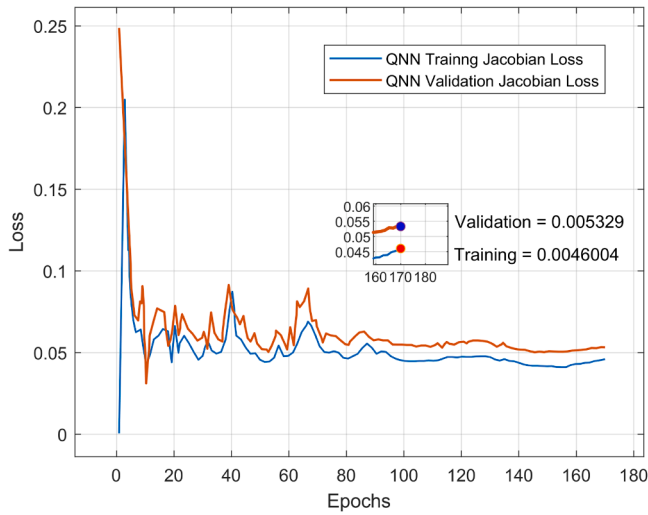


Fig. 14. Loss of QNN for singularity avoidance.

The other model investigated in this study uses the same MLP architecture and two different activation functions, with Swish used as a standard activation function in the ANN and the proposed quantum activation function in the QNN. The only difference is that rather than the MAE loss, the new loss function must consider a metric that expresses the distance from a singularity point and penalises short distances, as described in Eq. (12). We train the two networks over 170 training epochs, with the corresponding loss values shown in Figs. 13 and 14.

As shown in Fig. 13, neural networks commonly show a discernible trend during training. Typically, a notable decrease in loss is recorded during the initial epochs, after which the loss stabilises. This trend is visible in the training loss of the ANN, which exhibits oscillations but gradually decreases overall after an initial sharp drop. In the final epochs, as illustrated in the graph's inset, the training loss settles at around 0.005, while the validation loss is marginally higher at 0.006. This discrepancy between the two losses may indicate some degree of overfitting, where the model performs better on the training data than on unseen data; however, the similarity between these two values suggests that the model still has reasonable generalisation capacity, albeit with slight potential overfitting. The observed peaks in the training and validation losses may be due to the model's sensitivity to specific data points or batch sequences during training. Overall, the trend and final values indicate that the ANN can effectively learn the Jacobian-based function to avoid singularity points.

In the QNN model results shown in Fig. 14, the training process begins with a significant drop in loss and further rapid improvements in the initial epochs, which represents typical behaviour as the network begins to fit the training data. Subsequently, both training and validation losses exhibit an overall downward trend with some fluctuations. However, the validation loss closely tracks the training loss, suggesting the network is not overfitting and performing well on the new data. In the detailed view shown in the inset, the training loss reaches approximately 0.004 in the final epochs, with a slightly higher validation loss of 0.005. This minor gap between the training and validation losses indicates that the QNN can achieve consistent performance on unseen data, which is essential for ensuring the model's reliability in practical applications. The relatively low and comparable loss values indicate that the proposed QNN can successfully learn to approximate the Jacobian to avoid singularities both when exposed to the training data and when generalising to new data.

Comparing the two networks with singularity avoidance revealed that both models learnt the function adeptly. However, the QNN outperformed the ANN model by displaying a closer alignment between its

Table 4

Comparative analysis of the performance metrics for the proposed ANN and QNN models.

Model	Evaluation Metric	Loss (MAE) (Training / Validation)	Position Error (mm) (Training / Validation)	Orientation Error (radians) (Training / Validation)
ANN	MAE	0.0098 / 0.0109	–	–
QNN	MAE	0.0065 / 0.0072	–	–
ANN	Position Metrics	–	2.3 / 2.71	–
QNN	Position Metrics	–	1.76 / 1.79	–
ANN	Orientation Metrics	–	–	0.0198 / 0.021
QNN	Orientation Metrics	–	–	0.00176 / 0.00179
ANN	Singularity Avoidance MAE Jacobian	0.005 / 0.006	–	–
QNN	Singularity Avoidance MAE Jacobian	0.004 / 0.005	–	–
ANN	Position Metrics	–	1.72 / 1.97	–
QNN	Position Metrics	–	1.37 / 1.64	–

training and validation losses. The QNN demonstrated excellent accuracy in fitting the training data, with similar performance when tested on validation data. Overall, these findings indicate that the QNN is a more robust and superior regression model for accurately predicting outcomes in the studied problem. In the networks where the loss function prevents singularities, the validation position error is slightly lower than those without such avoidance measures; specifically, the validation position error rates for the ANN and QNN are 1.97 mm and 1.64 mm, respectively, compared to equivalent values of 2.71 mm and 1.79 mm for the ANN and QNN without singularity avoidance.

Table 4 summarises all the values obtained for the proposed networks in our study's training and validation phases. Overall, the QNN outperforms the ANN in terms of MAE and achieves superior values in the position and orientation metrics. In the validation step, the QNN's MAE is around 15.60 % lower than that of the ANN. Furthermore, the QNN exhibits superior precision in position and orientation errors, with differences of approximately 1 mm and 0.02 radians, respectively. The QNN's performance in all the evaluated metrics on the validation data is outstanding and superior, even in the network designed to avoid singularities. Compared with the ANN, the QNN achieves an MAE Jacobian that is roughly 16.67 % lower and a position error reduced by about 0.4 mm. The QNN with singularity avoidance also provides precise predictions, reducing the position error by approximately 0.15 mm compared to the QNN without singularity avoidance.

This study's proposed quantum-inspired activation function represents a considerable departure from traditional activation functions, providing improved learning capabilities and enhanced computational efficiency. This breakthrough tackles the long-standing issues of non-unique solutions and computational complexity in inverse kinematics. By showcasing the performance of QNNs, this study provides a basis for further exploration of implementing quantum principles in neural network models, potentially transforming various fields within robotics and beyond.

This research highlights the exceptional proficiency of the classical neural network model, which utilises the Swish activation function, in terms of training and validation accuracy. This model displays a rapid learning period and stabilises with low MAE values, demonstrating strong learning and generalisation capabilities. Introducing Gaussian noise further enhances the model's generalisation ability, resulting in improved performance.

Integrating quantum-inspired concepts into the neural network's activation function can significantly enhance the network's performance

and accuracy. The proposed QNN model displays more immediate learning and superior stabilisation compared to the classical ANN model. This superiority is demonstrated by the QNN's lower position and orientation error values, showcasing its superior precision and generalisation ability.

In this analysis, we also investigated models designed to prevent singularities, which is an essential aspect of robotics. The QNN model outperforms the classical model, with closer alignment between the training and validation loss values and lower position errors. This result indicates the QNN's robustness and superior regression capabilities, making it an excellent choice for real-world applications requiring precise positional estimations and reliable performance.

QNNs can outperform ANNs due to the advantages provided by quantum computing. One particularly important factor is parallelism, allowing the simultaneous processing of multiple possibilities through quantum superposition. This capability is particularly beneficial for complex mathematical problems in engineering, resulting in faster processing times and improved simulations. Quantum entanglement further enhances the efficiency of QNNs, allowing information to be processed across different parts of the network simultaneously. This effect improves the efficiency of algorithms for solving large systems of equations or matrix operations, which are common in engineering tasks. Quantum annealing also contributes to the superior performance of QNNs, enabling effective navigation of solution spaces and avoidance of local minima, which can trap classical algorithms. This feature is helpful in optimisation problems, where it is necessary to identify the most efficient design or lowest energy configuration.

QNNs are revolutionising how we address complex engineering challenges, such as predicting inverse kinematics for a six-degree-of-freedom robotic arm. By integrating quantum computing principles with neural networks, QNNs offer improved efficiency, performance, and accuracy compared to conventional neural networks. The progress presented in this study underscores the potential of quantum-inspired models to enhance the precision and performance of neural networks in robotics applications; however, we have yet to fully explore the comprehensive capabilities of QNNs as the field of quantum computing continues to evolve.

4. Conclusion

The proposed quantum-inspired activation function improves the learning efficiency and accuracy of neural networks, providing a reliable solution to the existing challenges of conventional methods. The findings of this study highlight the potential of quantum computing principles in refining neural network models, resulting in more accurate and efficient robotic systems.

Precise positioning and movement during complex tasks are essential for robotic arms, making inverse kinematics a critical component. However, classical approaches often need assistance in terms of handling non-unique solutions and increased computational complexity, especially with increased degrees of freedom. Therefore, innovative solutions are required to improve the accuracy and performance of these models and simplify calculations. While machine learning and artificial neural networks have shown promise in solving inverse kinematic equations, there is still a pressing need for further advancements, particularly in terms of applying quantum computing. By integrating quantum-inspired activation functions into the proposed neural network, this research aims to address these limitations and enhance the capabilities of robotic systems, contributing significantly to the field.

In terms of practical implications, the proposed QNN-based solution can enhance productivity and reduce error rates in manufacturing, assembly, welding, and painting tasks by improving the precision and computational efficiency of solving inverse kinematics for robotic arms. Avoiding singularities ensures consistent and reliable performance, which is crucial for maintaining smooth operations in complex industrial processes. Additionally, the enhanced precision and efficiency of

QNNs can significantly benefit applications in healthcare, such as robot-assisted surgery, and space exploration, where precise and efficient robotic manipulations are vital for mission success. Our examination addresses essential gaps in current industrial applications and demonstrates the potential for QNNs to revolutionise robotic control systems.

This research focuses on the kinematic modelling of the ABB IRB140 robot. It utilises the DH method for forward kinematics while defining the spatial geometry of a kinematic chain to determine the position and orientation of the robot arm. The Jacobian matrix is also essential to determine the mechanism's state and avoid the singularities described above. MLP neural networks were selected in this study to solve inverse kinematics. This work emphasises the importance of determining the optimal MLP structure and applying a robust training dataset. Additionally, the study presents quantum computing fundamentals and proposes an activation function for QNNs to enhance the accuracy of robotic arm motion predictions.

The results from the MLP demonstrate exceptional accuracy in predicting the inverse kinematics of the robot arm, as evidenced by the position and orientation errors in both training and validation. This result confirms that the network design was correct and the optimal parameter range was identified during hyperparameter selection. Second, the study highlights the superior performance of QNNs over standard ANNs in solving inverse kinematics for robotic arms. The QNNs indicated excellent precision in position and orientation errors, both with and without considering singularity avoidance. Specifically, the QNNs without singularity avoidance achieved a position error of 1.64 mm and a minor orientation error of 0.00179 radians.

These results demonstrate the potential of QNNs in enhancing the precision and efficiency of robotic arm manipulations, offering a promising avenue for future research in robotics and quantum computing. Quantum computing provides superior performance and accuracy by leveraging its unique advantages, such as parallelism and quantum entanglement. This technique enables the simultaneous processing of multiple possibilities, resulting in faster processing times and improved simulations for complex mathematical problems in engineering. Additionally, quantum annealing helps avoid local minima that can trap classical algorithms, effectively navigating solution spaces. These benefits may lead to more efficient algorithms for solving large systems of equations or matrix operations, which are common in engineering tasks that require the most efficient design or lowest energy configuration to be identified.

Overall, the results of this study demonstrate that QNNs outperform classical ANNs in terms of accuracy and robustness, particularly in scenarios involving singularity avoidance. However, there is room for further exploration and enhancement. Future research could include increasing the resolution and accuracy of the data collection process, conducting further hyperparameter optimisation and network architecture refinements, and developing further innovations in quantum methods and testing them in new applications. We acknowledge the importance of examining the performance of our models on different robotic arms and refining the novel quantum activation function in future research. This will allow us to further validate and generalise our findings across various robotic platforms and configurations, both in terms of actual robots and simulation models. Additionally, future research should focus on integrating more advanced quantum computing algorithms to improve the learning speed and adaptability of QNNs. Exploring the effects of different noise models on network accuracy and extending the application of QNNs to other domains in robotics, such as navigation and manipulation, will also be essential steps in developing the research capabilities and resources in this field.

Funding

This research was supported by funding from NSERC Canada.

CRediT authorship contribution statement

Mehdi Fazilat: Visualization, Methodology, Writing – original draft, Validation, Investigation, Formal analysis, Data curation, Conceptualization. **Nadjet Zioui:** Writing – review & editing, Writing – original draft, Validation, Supervision, Software, Resources, Project administration, Methodology, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

All data used to perform this work are available within the manuscript.

References

- [1] R.C. Jesus, E.O. Freire, L. Molina, E.A. Carvalho, Solving inverse kinematics of planar manipulators with robustness to singularities by reverse operating neural networks, in: 2018 Latin American Robotic Symposium, 2018 Brazilian Symposium on Robotics (SBR) and 2018 Workshop on Robotics in Education (WRE), IEEE, 2018, pp. 153–157.
- [2] M. Theofanidis, S.I. Sayed, J. Cloud, J. Brady, F. Makedon, Kinematic estimation with neural networks for robotic manipulators. *Artificial Neural Networks and Machine Learning-ICANN 2018: 27th International Conference On Artificial Neural Networks*, Rhodes, Greece, October 4-7, 2018, Proceedings, Part III 27, Springer, 2018, pp. 795–802.
- [3] R. Grassmann, V. Modes, J. Burgner-Kahrs, Learning the forward and inverse kinematics of a 6-DOF concentric tube continuum robot in SE (3), in: 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, 2018, pp. 5125–5132.
- [4] C.-K. Ho, C.-T. King, Automating the learning of inverse kinematics for robotic arms with redundant dofs, *arXiv preprint arXiv:2202.07869*, (2022).
- [5] R. Bensadoun, S. Gur, N. Blau, L. Wolf, Neural inverse kinematic, in: *International Conference on Machine Learning*, PMLR, 2022, pp. 1787–1797.
- [6] M.N. Vu, F. Beck, M. Schwegel, C. Hartl-Nesic, A. Nguyen, A. Kugi, Machine learning-based framework for optimally solving the analytical inverse kinematics for redundant manipulators, *Mechatronics* 91 (2023) 102970.
- [7] G. Fang, Y. Tian, Z.-X. Yang, J.M. Geraedts, C.C. Wang, Efficient jacobian-based inverse kinematics with sim-to-real transfer of soft robots by learning, *IEEE/ASME Trans. Mechatron.* 27 (2022) 5296–5306.
- [8] N.G. Adar, Real time control application of the robotic arm using neural network based inverse kinematics solution, *Sak. Univ. J. Sci.* 25 (2021) 849–857.
- [9] B.E. Hargis, W.A. Demirjian, M.W. Powelson, S.L. Canfield, Investigation of neural-network-based inverse kinematics for a 6-DOF serial manipulator with non-spherical wrist. *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, American Society of Mechanical Engineers, 2018. V05BT07A048.
- [10] A. Csizsar, J. Eilers, A. Verl, On solving the inverse kinematics problem using neural networks, in: 2017 24th International Conference on Mechatronics and Machine Vision in Practice (M2VIP), IEEE, 2017, pp. 1–6.
- [11] P. Srisuk, A. Sento, Y. Kitjaidure, Forward kinematic-like neural network for solving the 3D reaching inverse kinematics problems, in: 2017 14th International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology (ECTI-CON), IEEE, 2017, pp. 214–217.
- [12] V. Aparanji, U.V. Wali, R. Aparna, Robotic motion control using machine learning techniques, in: 2017 International Conference on Communication and Signal Processing (ICCSPP), IEEE, 2017, pp. 1241–1245.
- [13] A. Zlokap, California institute of technology, 2021.
- [14] S. Suneel, A. Balam, M. Amina Begum, K. Umapathy, P.C.S. Reddy, V. Talasila, Quantum mesh neural network model in precise image diagnosing, *Opt. Quantum Electron.* 56 (2024) 559.
- [15] R.D. Taylor, Quantum artificial intelligence: a “precautionary” US approach? *Telecomm. Policy* 44 (2020) 101909.
- [16] S.T. Marella, H.S.K. Parisa, Introduction to quantum computing, *quantum computing and communications*, (2020).
- [17] S.B. Ramezani, A. Sommers, H.K. Manchukonda, S. Rahimi, A. Amirlatifi, Machine learning algorithms in quantum computing: a survey, in: 2020 International joint conference on neural networks (IJCNN), IEEE, 2020, pp. 1–8.
- [18] N.H. Nguyen, E.C. Behrman, M.A. Moustafa, J.E. Steck, Benchmarking neural networks for quantum computations, *IEEE Trans. Neural Netw. Learn. Syst.* 31 (2019) 2522–2531.
- [19] S. Gupta, S. Mohanta, M. Chakraborty, S. Ghosh, Quantum machine learning-using quantum computation in artificial intelligence and deep neural networks: quantum computation and machine learning in artificial intelligence, in: 2017 8th annual industrial automation and electromechanical engineering conference (IEMECON), IEEE, 2017, pp. 268–274.
- [20] M. Zidan, A. Sagheer, N. Metwally, Autonomous perceptron neural network inspired from quantum computing, *arXiv preprint arXiv:1510.00556*, (2015).
- [21] A. Kutvonen, K. Fujii, T. Sagawa, Optimizing a quantum reservoir computer for time series prediction, *Sci. Rep.* 10 (2020) 14687.
- [22] M.P. Singh, B. Rajput, Role of entanglement in quantum neural networks (QNN), *J. Mod. Phys.* 6 (2015) 1908–1920.
- [23] M. Fazilat, N. Zioui, J. St-Arnaud, A novel quantum model of forward kinematics based on quaternion/pauli gate equivalence: application to a six-jointed industrial robotic arm, *Results Eng.* 14 (2022) 100402.
- [24] N. Zioui, Y. Mahmoudi, A. Mahmoudi, M. Tadjine, S. Bentouba, A novel quantum-computing-based quaternions model for a robotic arm position, *Int. J. Comput. Intell. Control* 13 (2021) 71–77.
- [25] N. Zioui, A. Mahmoudi, M. Tadjine, Representing quantum spins in different coordinate systems for modelling rigid body orientation, *Karbala Int. J. Mod. Sci.* 9 (2023) 11.
- [26] H.M. Abdulridha, Z.A. Hassoun, Control design of robotic manipulator based on quantum neural network, *J. Dyn. Syst. Meas. Control* 140 (2018) 061002.
- [27] K. Takahashi, M. Tsuji, M. Hashimoto, Remarks on Learning Inverse Kinematics of a Robot Manipulator Using a Quaternion Neural Network, In: 10th IIAE International Conference on Industrial Application Engineering, 2022, doi: 10.12792/iciae2022.011.
- [28] S. Dereli, R. Köker, A meta-heuristic proposal for inverse kinematics solution of 7-DOF serial robotic manipulator: quantum behaved particle swarm algorithm, *Artif. Intell. Rev.* 53 (2020) 949–964.
- [29] X. Cheng, M. Zhao, The inverse solution algorithm and trajectory error analysis of robotic arm based on MQACA-RBF network, *J. Robot.* 2020 (2020) 7807952.
- [30] L. Parisi, D. Neagu, R. Ma, F. Campean, QReLU and m-QReLU: two novel quantum activation functions to aid medical diagnostics, *arXiv preprint arXiv:2010.08031*, (2020).
- [31] Y. Dong, F. Li, T. Zhu, R. Yan, Air quality prediction based on quantum activation function optimized hybrid quantum classical neural network, *Front. Phys.* 12 (2024) 1412664.
- [32] A. Daskin, A simple quantum neural net with a periodic activation function, in: 2018 IEEE International Conference on Systems, Man, and Cybernetics (SMC), IEEE, 2018, pp. 2887–2891.
- [33] S. Li, M.S. Salek, Y. Wang, M. Chowdhury, Quantum-inspired activation functions in the convolutional neural network, *arXiv preprint arXiv:2404.05901*, (2024).
- [34] Y. She, S. Li, M. Xin, Quantum-interference artificial neural network with application to space manipulator control, *IEEE Trans. Aerosp. Electron. Syst.* 57 (2021) 2167–2182.
- [35] V.S. Gandhi, T.-M. McGinnity, Quantum neural network based surface EMG signal filtering for control of robotic hand, in: The 2013 International Joint Conference on Neural Networks (IJCNN), IEEE, 2013, pp. 1–7.
- [36] M. Fazilat, N. Zioui, The impact of simplifications of the dynamic model on the motion of a six-jointed industrial articulated robotic arm movement, *J. Robot. Control (JRC)* 5 (2024) 173–186.
- [37] M. Fazilat, Université du Québec à trois-rivières, 2022.
- [38] S. Sarabandi, F. Thomas, Solution methods to the nearest rotation matrix problem in $\mathbb{R}^3$: a comparative survey, *Numer. Linear Algebra Appl.* 30 (2023) e2492.
- [39] G. Giarmatzis, E.I. Zacharaki, K. Moustakas, Real-time prediction of joint forces by motion capture and machine learning, *Sensors* 20 (2020) 6933.
- [40] M.d.R. Martínez-Blanco, T. Ibarra-Pérez, F. Olivera-Domingo, J.M. Ortiz-Rodríguez, Optimization of Training Data Set Based On Linear Systematic Sampling to Solve the Inverse Kinematics of 6 DOF Robotic Arm With Artificial Neural Networks, *Frontiers of Data and Knowledge Management For Convergence of ICT, Healthcare, and Telecommunication Services*, 2022, pp. 85–112.
- [41] S.B. Šegota, N. Anđelić, V. Mrzljak, I. Lorencin, I. Kuric, Z. Car, Utilization of sublayer perceptron for determining the inverse kinematics of an industrial robotic manipulator, *Int. J. Adv. Robot. Syst.* 18 (2021) 1729881420925283.
- [42] R. Liu, F. Nageotte, P. Zanne, M. de Mathelin, B. Dresp-Langley, Deep reinforcement learning for the control of robotic manipulation: a focussed mini-review, *Robotics* 10 (2021) 22.
- [43] M.A. Funes Lora, E.A. Portilla-Flores, R. Rivera Blas, E.A. Merchan Cruz, M. F. Carbajal Romero, Metaheuristic techniques comparison to optimize robotic end-effector behavior and its workspace, *Int. J. Adv. Robot. Syst.* 15 (2018) 1729881418801132.
- [44] ““(0) = “!count(descendant::ce:intra-ref)”“(0) = “!count(descendant::ce:doi)”“(0) [?pdfstart “Link”] > R. Singh, V. Kukshal, V.S. Yadav, A review on forward and inverse kinematics of classical serial manipulators, *Adv. Eng. Des.* 2020 (2021) 417–428.
- [45] A. Rosyid, B. El-Khasawneh, A. Alazzam, Performance measures of parallel kinematics manipulators, *Mech. Sci.* 11 (2020) 49–73.
- [46] R. Liu, E. Dobriban, Z. Hou, K. Qian, Dynamic load identification for mechanical systems: a review, *Arch. Comput. Methods Eng.* 29 (2022) 831–863.
- [47] S. Sarabandi, F. Thomas, A survey on the computation of quaternions from rotation matrices, *J. Mech. Robot.* 11 (2019) 021006.
- [48] Y. Lu, A. Sigov, L. Ratkin, L.A. Ivanov, M. Zuo, Quantum computing and industrial information integration: a review, *J. Ind. Inf. Integr.* (2023) 100511.
- [49] N. Zioui, A. Mahmoudi, Y. Mahmoudi, M. Tadjine, Quantum computing based state domain equations and feedback control, *Results Appl. Math.* 19 (2023) 100385.
- [50] H. Feraoun, M. Fazilat, R. Dermouche, S. Bentouba, M. Tadjine, N. Zioui, Quantum maximum power point tracking (QMPPPT) for optimal solar energy extraction, *Syst. Soft Comput.* (2024) 200118.

- [51] S. Saidat, R. Boumekhita, M. Tadjine, N. Zioui, Quantum pulse-width modulation design and implementation for a DC motor drive, *Quantum Inf. Process.* 23 (2024) 88.
- [52] G. Czelusta, J. Mielczarek, Quantum simulations of a qubit of space, *Phys. Rev. D* 103 (2021) 046001.
- [53] A. Usha Devi, Sudha, I. Reena, H. Karthik, A. Rajagopal, Quantum correlations in symmetric multiqubit systems, *J. Indian Inst. Sci.* 103 (2023) 419–447.
- [54] S. Bentouba, N. Zioui, Y. Mahmoudi, A. Mahmoudi, M. Tadjine, A new quantum-computing-based algorithm for robotic arms and rigid bodies, (2021).
- [55] Y. Mahmoudi, N. Zioui, H. Belbachir, Quantum optimization in engineering: a brief review, in: 2023 International Conference on Decision Aid Sciences and Applications (DASA), IEEE, 2023, pp. 23–27.
- [56] D. Koutný, L. Ginés, M. Moczala-Dusanowska, S. Höfling, C. Schneider, A. Predojević, M. Ježek, Deep learning of quantum entanglement from incomplete measurements, *Sci. Adv.* 9 (2023) eadd7131.
- [57] M. Fazilat, N. Zioui, Quantum-inspired sliding-mode control to enhance the precision and energy efficiency of an articulated industrial robotic arm, *Robotics* 14 (2025) 14.
- [58] Y. Mahmoudi, N. Zioui, H. Belbachir, M. Tadjine, A. Rezgui, A brief review on mathematical tools applicable to quantum computing for modelling and optimization problems in engineering, *Emerg. Sci. J.* 7 (2022) 289–312.
- [59] M. Fazilat, N. Zioui, Investigating quantum artificial neural networks for singularity avoidance in robotic manipulators, in: 2024 12th International Conference on Systems and Control (ICSC), IEEE, 2024, pp. 335–340.
- [60] D. Reda, A. Talaoubrid, M. Fazilat, N. Zioui, M. Tadjine, A quantum direct torque control method for permanent magnet synchronous machines, *Comput. Electr. Eng.* 122 (2025) 109994.
- [61] N. Zioui, A. Mahmoudi, M. Tadjine, Design of a new hybrid linearising-backstepping controller using quantum state equations and quantum spins, *Int. J. Autom. Control* 17 (2023) 397–417.
- [62] M. Weigold, J. Barzen, F. Leymann, M. Salm, Encoding patterns for quantum algorithms, *IET Quantum Commun.* 2 (2021) 141–152.
- [63] C. Zhao, X.-S. Gao, QDNN: DNN with quantum neural network layers, *arXiv preprint arXiv:1912.12660*, (2019).
- [64] A. Shukla, P. Vedula, An efficient quantum algorithm for preparation of uniform quantum superposition states, *Quantum Inf. Process.* 23 (2024) 38.
- [65] N. Killoran, T.R. Bromley, J.M. Arrazola, M. Schuld, N. Quesada, S. Lloyd, Continuous-variable quantum neural networks, *Phys. Rev. Res.* 1 (2019) 033063.
- [66] E.P. Tapia, G. Scarpa, A. Pozas-Kerstjens, A didactic approach to quantum machine learning with a single qubit, *Phys. Scr.* 98 (2023) 054001.
- [67] R. Vale, T.M.D. Azevedo, I. Araújo, I.F. Araujo, A.J. da Silva, Decomposition of multi-controlled special unitary single-qubit gates, *arXiv preprint arXiv:2302.06377*, (2023).